

Grundlagen Theoretischer Informatik

by Barbara Hammer

Summer semester 2026

Lecture notes by Liliana Sanfilippo

April 2026

Inhaltsverzeichnis

1	Formale Sprachen und Grammatiken	5
1.1	Historisches	5
1.2	Kapitelinhalt	5
1.3	Wortersetzungssysteme und Notationen	6
1.4	Reguläre Sprachen	8
1.4.1	Lineare Grammatiken	8
1.4.2	Strukturelle Eigenschaften	9
1.5	Reguläre Sprachen	9
1.6	Kontextfreie Sprachen	9
2	Weiteres	11
2.1	Klausurstruktur	11
2.1.1	Sprachunterscheidung	11
2.1.2	Endliche Automaten	13
2.1.3	Kontextfreie Grammatiken & LR/Kellerautomaten	15
2.1.4	Probleme / Berechenbarkeit & Reduktion	17
2.1.5	Multiple-Choice	17
	Backmatter	i
	Index	i
	Definitions	i
	References	iii

KAPITEL 1

Formale Sprachen und Grammatiken

“Formale Sprachen referieren im Gegensatz zu natürlichen Sprachen auf Sprachen in der Technik: wie unterhalte ich mich mit einem Rechner oder Roboter? Eins der wichtigsten Beispiele sind Programmiersprachen auf unterschiedlichem Niveau, angefangen von Assembler über gängige Hochsprachen wie Java oder C++ zu Anwendungssprachen wie etwa Matlab. Andere wichtige Bereiche, in denen formale Sprachen eine zentrale Rolle spielen, sind etwa XML, oder aber direkte Maschinenbefehle etwa in der NC Steuerung. Anders als natürliche Sprache muss dieses so präzise sein, dass die Maschine keinerlei Interpretationsspielraum hat und genau das versteht, was der Mensch intendiert. Zusätzliche Anforderung ist, dass dieses auch möglichst effizient umgesetzt werden kann, d.h. der Rechner sollte ein Wort möglichst schnell erkennen und interpretieren können.” [1]

0_orga.pdf

Vorerst irrelevant.

1.1 | Historisches

1_theinf_motivation.pdf - Skript S. 1–2

“Die Formalisierung von Sprache ist ein nicht nur für die Informatik im Rahmen technischer Systeme relevantes Gebiet, sondern auch im Rahmen der Sprachphilosophie und Linguistik behandelt worden. So ist einer der wichtigsten Forscher in dem Gebiet Noam Chomsky, ein Linguist, der 1956 die nach ihm benannte Chomsky-Hierarchie von formalen Sprachen einführte (Chomsky 1956). Diese unterscheidet unterschiedlich komplexe Formalisierungen, die zu unterschiedlich komplexen Sprachkonstrukten führen. Diese theoretischen Überlegungen hatten in der Informatik direkte Bedeutung für die Entwicklung von Programmiersprachen. Die erste Programmiersprache, für die eine formale Syntax eingeführt wurde, war um 1960 ALGOL 60. Unter anderem war hier John Backus beteiligt, nach dem die heute noch gebräuchliche Backus-Naur Form von Grammatiken benannt ist. Basierend auf dieser Formalisierung war die Entwicklung von Compilern deutlich einfacher und eleganter möglich.

Das Gebiet der Formalen Sprachen behandelt unter anderem auch, was auf unterschiedlichen Komplexitätsstufen darstellbar ist und welche strukturellen Eigenschaften gelten. Interessanterweise sind einige wichtige strukturelle Eigenschaften in dem Gebiet auch noch offen bzw. teilweise sehr spät gelöst worden. Der Abschluss so genannter kontextsensitiver Sprachen gegen Komplementbildung wurde erst 1988 bewiesen im Satz von Immerman/Szelepcsényi (Immerman 1988). Letzterer war damals noch Student. Er erhielt 1995 zusammen mit Immerman hierfür den Gödel-Award, den Nobelpreis der Informatik.” [1]

1.2 | Kapitelinhalt

2_0_sprachen_intro.pdf - Skript S. 2

“In diesem Kapitel werden wir eine Hierarchie von formalen Sprachen auf der Basis von Grammatiken und deren Komplexität kennenlernen. Das dient dazu, unterschiedlich komplexe formale Sprachen definieren und interessante Eigenschaften wie etwa den Abschluss untersuchen zu können.” [1]

1.3 | Wortersetzungssysteme und Notationen

2_1_Sprachen_Grundlagen.pdf - Skript S. 2–6

“Wortersetzungssysteme oder auch Semi-Thue-Systeme bilden ein grundlegendes Konzept, Wörter zu erzeugen oder abzuändern. Ausgehend von einem Startsymbol kann man eine Menge komplexer Wörter durch entsprechende Ersetzungsregeln definieren. Wir werden Wortersetzungssysteme als Grundlage in der gesamten Vorlesung verwenden. Zunächst geben wir einige Definitionen:” [1]

Definition 1.1 Symbol todo.

Definition 1.2 Alphabet Endliche Menge von Symbolen:

$$\Sigma = \{a_1, \dots, a_n\}$$

Dabei ist Σ^* die Menge aller Wörter über Σ und Σ^m die Menge aller Wörter der Länge m über Σ .

Definition 1.3 Wort Eine endliche Folge von Elementen aus einem Alphabet:

$$\bar{w} = a_{i_1} \dots a_{i_m}, \quad a_{i_j} \in \Sigma, \quad m > 0$$

Dabei ist $m = \bar{w}$ die Länge des Wortes und ein Wort der Länge 0 ist das leere Wort ϵ .

Definition 1.4 Anzahl der Vorkommnisse des Symbols in einem Wort Die Anzahl der Vorkommnisse des Symbols a_i im Wort $\bar{w}$.

Definition 1.5 Leeres Wort Wort der Länge 0, notiert durch ϵ .

Definition 1.6 Σ^* Menge aller Wörter über Σ .

Definition 1.7 Σ^m Menge der Wörter der Länge m über Σ .

Definition 1.8 Konkatenation Eine Verkettung zweier Wörter $\bar{w} = a_{i_1} \dots a_{i_m}$ und $\bar{v} = a_{i_1} \dots a_{i_l}$ zu

$$\bar{w}\bar{v} = a_{i_1} \dots a_{i_m} a_{i_1} \dots a_{i_l}$$

.

Definition 1.9 Präfix $\bar{w}$ von $\bar{u}$ Wenn $\bar{u} = \bar{w}\bar{v}$.

Definition 1.10 Postfix $\bar{v}$ von $\bar{u}$ Wenn $\bar{u} = \bar{w}\bar{v}$.

Definition 1.11 Monoid Assoziative Strukturen, die ein neutrales Element haben und für die eine assoziative Operation mit Einselement existiert.

Satz 1.1 Aus der Gleichheit $\overline{uv} = \overline{u}\overline{v}$ oder $\overline{vu} = \overline{v}\overline{u}$ folgt $\overline{v} = \overline{u}$

Notation 1.1 Potenzen von Wörtern $\overline{w}$ schreiben wir kurz als $\overline{w}^k = \underbrace{\overline{w} \dots \overline{w}}_{k \text{ mal}}$.

Beispiel 1.1 Zu Definition 1.2 (Beispiel 2 in Skript) $\{0, 1\}$, $\{a, b, c\}$, $\{\underline{4711}, \underline{ha}, \underline{ende}\}$ sind Alphabets, wobei jeder Eintrag als ein untrennbares Zeichen gewertet wird.

$\{0, \underline{1}, \underline{2}, \underline{3}, \dots\}$, ist kein gültiges Alphabet, weil die Menge unendlich groß ist.

$\{a, \underline{aa}, \underline{aaa}\}$ ist zwar ein gültiges Alphabet nach unserer Definition, aber nicht empfehlenswert, weil es keine eindeutige Identifikation der Zeichen erlaubt.

Beispiel 1.2 Zu Definition 1.7 Σ^m (Beispiel 2 in Skript) Alle Wörter über dem Alphabet $\{0, 1\}$ der Länge maximal 3, aufgeschrieben in lexikographischer Reihenfolge, sind

$\epsilon, 0, \underline{1}, \underline{00}, \underline{01}, \underline{10}, \underline{11}, \underline{000}, \underline{001}, \underline{010}, \underline{100}, \underline{011}, \underline{110}, \underline{101}, \underline{111}$

Notation 1.2 “Um besser zwischen Variablen und Konstanten zu unterscheiden, schreiben wir konstante Wörter and Symbole unterstrichen und aufrecht (etwa **bla**). Variablen schreiben wir stattdessen kursiv. Außerdem heben wir Wortvariablen, ähnlich zu Vektoren in der linearen Algebra, durch einen Überstrich hervor (etwa $\overline{w}$).” [1]

Definition 1.12 Wortersetzungssystem über einem Alphabet Σ Eine endliche Menge R von Ersetzungsregeln über Σ type

Bemerkung 1.1 “Für den Begriff “Ersetzungsregel” gibt es reichlich Synonyme, z.B. “Regel”, “Grammatikregel”, “Substitution”, “Produktionsregel” und “Produktion”. Dementsprechend wird ein Wortersetzungssystem auch häufig “Regelmenge” oder “Produktionen” genannt” [1]

Definition 1.13 Ersetzungsregel über einem Alphabet Σ Ein Tupel von Worten $\overline{\alpha}, \overline{\beta} \in \Sigma^*$, geschrieben als $\overline{\alpha} := \overline{\beta}$

Definition 1.14 Wort $\overline{v} \in \Sigma^*$ in einem Schritt ableitbar aus Wort $\overline{u} \in \Sigma^*$ Falls es eine Ersetzungsregel über einem Alphabet Σ $\overline{\alpha} := \overline{\beta}$ in R gibt mit $\overline{u} = \overline{u_1\alpha u_2}$ und $\overline{v} := \overline{u_1\beta u_2}$. Das heißt, $\overline{v}$ erhält man aus $\overline{u}$, indem man das gemäß einer Ersetzungsregel das Teilwort $\overline{\alpha}$ mit dem Teilwort $\overline{\beta}$ ersetzt.
Schreibweise: $\overline{u} \rightarrow_R^1 \overline{v}$ oder $\overline{u} \rightarrow_R \overline{v}$

Bemerkung 1.2 Falls das Wortersetzungssystem für die Ableitung aus dem Kontext klar ist, schreiben wir $\rightarrow_R$ auch als $\rightarrow$

Diese Relation gibt Anlass zu ihrer transitiven Hülle der in $t \in \mathbb{N}$ Schritten auseinander ableitbaren Wörter. Wir definieren:

Definition 1.15 in $t \in \mathbb{N}$ Schritten auseinander ableitbare Wörter todo

Die transitive Hülle ist dann gegeben als die Menge

$$\{(\bar{u}, \bar{v}) \mid \bar{u}\}$$

Notation 1.3 Falls für irgendein

Definition 1.16 Aus $\bar{u}$ ableitbare Wörter $\bar{u} \rightarrow_R^0 \bar{v} \Leftrightarrow \bar{u} = \bar{v}$ und $\bar{u} \rightarrow_R^{t+1} \bar{v} \Leftrightarrow \exists \bar{u}' \in \Sigma^*, \text{ mit } \bar{u} \rightarrow_R^1 \bar{u}', \bar{u}' \rightarrow_R^t \bar{v}.$

Definition 1.17 Formale Sprache todo.

Definition 1.18 Grammatik todo.

Definition 1.19 Erzeugte Sprache todo.

Definition 1.20 Äquivalenz todo.

Definition 1.21 Formale Regelsprache todo.

1.4 | Reguläre Sprachen

Definition 1.22 Verkettung todo.

Definition 1.23 Kleene-Stern todo.

Definition 1.24 Potenz todo.

Definition 1.25 Regulärer Ausdruck todo.

Konvention 1.1 Wie in anderen Programmiersprachen üblich vereinbaren wir noch ein paar Kurzformen. Sei A ein regulärer Ausdruck über dem Alphabet $\Sigma = \{a_1, \dots, a_n\}$, dann ist:

Definition 1.26 Reguläre Sprache todo.

“reguläre Ausdrücke werden bei der Textverarbeitung häufig benutzt. Es ist daher interessant, möglichst einfache reguläre Ausdrücke für eine gegebene Sprache zu konstruieren. Ein Maß, das hier betrachtet wird, ist die so-genannte Sternhöhe, also die Anzahl ineinander verschachtelter Operatoren $*$. Das Sternhöhenproblem war tatsächlich bis 1988 noch ungelöst; das ist die Frage, für eine gegebene Sprache einen regulären Ausdruck zu finden mit möglichst wenig ineinander verschachtelten Operatoren $*$. ” [1]

1.4.1 | Lineare Grammatiken

Definition 1.27 Lineare Grammatik todo.

Definition 1.28 Rechtslineare Grammatik todo.

Definition 1.29 Linkslinare Grammatik todo.

“Kommen wir zu einem “wüsten Beweis”, der reguläre Ausdrücke zu linkslinearen Grammatiken in Beziehung setzt (ein äquivalenter Beweis würde es erlauben, rechtslineare Grammatiken zu regulären Ausdrücken in Beziehung zu setzen, die Begriffe sind tatsächlich äquivalent):” [1]

Satz 1.2 Eine Sprache L über Σ ist regulär dann und nur dann, wenn es eine rechtslineare Grammatik G gibt mit $L(G) = L$.

Beweise Skript Seite 9–11.

1.4.2 | Strukturelle Eigenschaften

Für reguläre Sprachen gelten wichtige strukturelle Eigenschaften:

Satz 1.3 Satz über die strukturellen Eigenschaften con regulären Sprachen

reguläre Sprachen sind gegen Vereinigung, Schnitt, Konkatenation, Komplement und Kleene-Stern abgeschlossen, d.h. sind L und L' regulär, dann auch $L \cup L'$, $L \cap L'$, $L \circ L'$, $\Sigma^* \setminus L$ und L^* .

Beweis. Skript Seite 13.

Satz 1.4 Pumping-Lemma

todo

Beweis. Skript Seite 13–14.

1.5 | Reguläre Sprachen

2_2_Sprachen_Regulaer_Part1.pdf - Skript S. 6–13

2_2_Sprachen_Regulaer_Part2.pdf bis F. 57 - Skript S. 13–15

1.6 | Kontextfreie Sprachen

2_2_extra_pumpinglemma_beispiele.pdf - Extra

2_2_Sprachen_Regulaer_Part2.pdf ab F. 58 - Skript S. 15–19

Satz 1.5 Pumping-Lemma

todo

Beweis. Skript Seite 13–14.

Beispiel 1.3 Zu Folie 7–8 aus 2_2_extra_pumpinglemma_beispiele.pdf

acbc abba ba

eine zerlegung ist:

$$\underbrace{acbc}_{u} \underbrace{abba}_{v} \underbrace{ba}_{w}$$

eine andere wäre:

$$\underbrace{ac}_u \underbrace{bc}_v \underbrace{abbaba}_w$$

Das schafft man mit dem pumpen nur wenn in einem regex oirgendwo ein Kleen Stern ist, alles vor dem stern ust das u, das wo der stern drauf angewendet weird ist v und das danach w. Wenn man sterne an mehrere Stellen setzen könnte, gibt es mehrere Zerlegungen.

Beispiel 1.4 Zu F 58 an Tafel

Gegeben:

$$L_1 := \{a^n b^n \mid n \geq 0\}$$

$$L_2 := \{x \in \{a, b\}^* \mid \#a = \#b\}$$

$$L_3 := \{a^{n^2} \mid n \geq 0\}$$

Jemand behauptet: L_2 ist nicht regulär, denn $L_1 \subseteq L_2$ und L_1 ist nicht regulär.

Stimmt das? Nein, falsch!

Klingt intuitiv, aber Teilmengenbeziehungen einer **regulären Sprache** sagt nichts über die Regularität der anderen beteiligten Sprache aus.

Stattdessen macht man das mit einem formalen Widerspruchsbeweis (F 60):

Pumping-Lemma beschreibt wie reg sprachen aussehen, wenn die unendlich groß werden

Wichtigster Effekt/Ziel der Vorlesung: Sie will uns Formalierungen schmackhaft machen

KAPITEL 2

Weiteres

2.1 | Klausurstruktur

Aufg.	Thema	Kernkompetenz
1	Sprachunterscheidung	Chomsky-Hierarchie, Pumping-Lemma, Grammatiken, RegExp
2	Endliche Automaten	NEA/DEA, Potenzmengenkonstr., Abschluss-Konstruktionen
3	Kontextfreie Gramm. & LR/Kellerautomaten	Eindeutigkeit, LR-Parsing, Strukturbaum, CYK, PDA
4	Probleme / Berechenbarkeit & Reduktion	P, NP, $\leq_p$, Reduktionen, Chomsky-Größen
5	Multiple Choice	Querschnitt über den ganzen Stoff

2.1.1 | Sprachunterscheidung

Zu einer Sprache L entscheiden und beweisen, in welche Klasse sie fällt (regulär / nicht regulär, kontextfrei / nicht kontextfrei). Meist 3-4 Teilaufgaben, die die vier Beweisrichtungen kombinieren.

Bemerkung 2.1 Fast immer Varianten von $\{a^n b^m c^k \mid \text{lineare (Un-)Gleichung über } n, m, k\}$

Vollständige Klassifikation (reg/nreg und cf/ncf) mit Beweis.

Für jede Sprache beide Fragen beantworten: nicht regulär $\Rightarrow$ Pumping-Lemma (regulär); kontextfrei $\Rightarrow$ CFG/PDA; nicht kontextfrei $\Rightarrow$ Pumping-Lemma (kontextfrei).

■ Ist die folgende Sprache regulär/nicht regulär, kontextfrei/nicht kontextfrei und warum:

- $\{\bar{x} \in \{a, b\}^* \mid \bar{x} = a^n b^n \text{ mit } n \in \mathbb{N}\} \cup \{\bar{x} \in \{c, d\}^* \mid |\bar{x}| \leq 42\}$ (HK2017)
- $\{\bar{z} \in \{a, b, c\}^* \mid \text{Anzahl}(a, \bar{z}) = \text{Anzahl}(b, \bar{z}) = \text{Anzahl}(c, \bar{z})\}$ (NK2017)
- $\{a^m b^n c^k \mid n, m, k \in \mathbb{N}_0, m = k\}$ (HK2019)
- $\{a^m b^n c^k \mid n, m, k \in \mathbb{N}_0, m \geq n \geq k\}$ (HK2019)
- $\{(ab)^n \mid n \in \mathbb{N}_0\}$ (HK2019)
- $\{a^m b^n \mid m + nk; m, n, k \in \mathbb{N}_0\}$ (NK2019)
- $\{a^m b^n c^k \mid n, m, k \in \mathbb{N}_0, m = n\}$ (NK2019)
- $\{a^m b^n c^k \mid n, m, k \in \mathbb{N}_0, m \leq n \leq k\}$ (NK2019)
- $\{\bar{x} \in \{a, b, c\}^* \mid \text{Anzahl}(a, \bar{x}) + \text{Anzahl}(b, \bar{x}) \text{ ist gerade} \wedge \text{Anzahl}(b, \bar{x}) + \text{Anzahl}(c, \bar{x}) \text{ ist ungerade}\}$ (PK2021)
- $\{a^m b^n c^k \mid n, m, k \in \mathbb{N}_0, n = m + k\}$ (PK2021)
- $\{\bar{x} \in \{a, \dots, z\}^* \mid \exists n : |\bar{x}| = 42^n\}$ (PK2021)

■ Ist die folgende Sprache regulär/nicht regulär, kontextfrei/nicht kontextfrei? Beweisen Sie Ihre Antwort

- ☐ $\{\underline{a}^n, \underline{b}^m, \underline{c}^k \mid n \leq m \leq k\}$ (PK2021)
- ☐ $\{a^n b^m c^k \mid n, m, k \in \mathbb{N}_0, n + m = k\}$ (PK2023)
- ☐ $\{a^n b^m c^k \mid n, m, k \in \mathbb{N}_0, n \leq m\} \cap \{a^n b^m c^k \mid n, m, k \in \mathbb{N}_0, m \leq k\}$ (PK2023)
- ☐ $\{a^n b^m \mid n, m, k \in \mathbb{N}_0, n + m = k, k > 5\}$ (HK2023)
- ☐ $\{a^m b^n c \mid n, m \in \mathbb{N}_0, n \leq m\} \cap \{a^n b^m c^k \mid n, m, k \in \mathbb{N}_0, n \leq m\}$ (HK2023)
- ☐ $\{a^n b^m c^k \mid n, m, k \in \mathbb{N}_0, n = k\} \cap \{a^m b^n c \mid n, m \in \mathbb{N}_0\}$ (PK2025)
- ☐ $\{a^{2025} b^{2025} c^k \mid k \in \mathbb{N}_0, k \leq 2025\}$ (PK2025)

Nicht-Regularität mit dem Pumping-Lemma zeigen.

Typische Sprachen: $\{a^n b^m c^k \mid n = m\}$, $\{a^{2n} b^{4n}\}$, $\{c^4 a^n b^n\}$. Gefragt ist explizite Anwendung, das Gegenspieler-Schema muss also vollständig sein.

■ Zeigen Sie durch explizite Verwendung des Pumpinglemmas, dass die folgende Sprache nicht regulär ist:

- ☐ $\{\underline{a}^{2n}, \underline{b}^{4n} \mid n \in \mathbb{N}\}$ (HK2017)
- ☐ $\{\underline{c}^4, \underline{a}^n, \underline{b}^n\}$ (NK2017)
- ☐ $\{a^n b^m c^k \mid n, m, k \in \mathbb{N}_0, n = k\}$ (PK2023)
- ☐ $\{a^n b^m c^k \mid n, m, k \in \mathbb{N}_0, n = m\}$ (NK2023)
- ☐ $\{a^n b^k c^m \mid n, m, k \in \mathbb{N}_0, n = k\}$ (PK2025)

Kontextfreiheit durch Angabe einer kontextfreien Grammatik zeigen.

Für L_1 aus dem Pumping-Teil wird meist direkt danach die passende CFG verlangt.

■ Geben Sie eine explizite kontextfreie Grammatik für L_1 (Teilaufgabe ...) an.

- ☐ $\{a^n b^m c^k \mid n, m, k \in \mathbb{N}_0, n = k\}$ (PK2023)
- ☐ $\{a^n b^m c^k \mid n, m, k \in \mathbb{N}_0, n = m\}$ (NK2023)
- ☐ $\{a^n b^k c^m \mid n, m, k \in \mathbb{N}_0, n = k\}$ (PK2025)

Regularität durch regulären Ausdruck (oder DEA/rechtslin. Grammatik) zeigen.

z. B. $\{x \in \{a, b\}^* \mid \#_a(x) \text{ gerade}\}$ oder $\#_a + \#_b \geq 2$. Konstruktives Vorgehen: RegExp konkret hinschreiben.

■ Zeigen Sie durch die Konstruktion eines regulären Ausdrucks, dass die folgende Sprache regulär ist

- ☐ $\{\bar{x} \in \{a, b\}^* \mid \text{Anzahl}(\underline{a}, \bar{x}) \text{ gerade}\}$ (HK2017)
- ☐ $\{\bar{x} \in \{a, b, c\}^* \mid \text{Anzahl}(\underline{a}, \bar{x}) = \text{Anzahl}(\underline{b}, \bar{x}) \geq 2\}$ (NK2017)

Schnitt/Vereinigung zweier Sprachen aka "erst den Schnitt berechnen".

- $\{\bar{x} \in \{a, b\}^* \mid \bar{x} = a^n b^n \text{ mit } n \in \mathbb{N}\} \cup \{\bar{x} \in \{c, d\}^* \mid |\bar{x}| \leq 42\}$ (HK2017)
- $\{a^n b^m c^k \mid n, m, k \in \mathbb{N}_0, n \leq m\} \cap \{a^n b^m c^k \mid n, m, k \in \mathbb{N}_0, m \leq k\}$ (PK2023)
- $\{a^m b^n c \mid n, m \in \mathbb{N}_0, n \leq m\} \cap \{a^n b^m c^k \mid n, m, k \in \mathbb{N}_0, n \leq m\}$ (HK2023)
- $\{a^n b^m c^k \mid n, m, k \in \mathbb{N}_0, n = k\} \cap \{a^m b^n c \mid n, m \in \mathbb{N}_0\}$ (PK2025)

Grammatik für Vereinigung/Konkatenation zweier Sprachen.

Allgemeine Konstruktion $G_{\cup}$ bzw. $G_{\bullet}$ aus zwei gegebenen Grammatiken angeben.

- Seien G und G' rechtslineare Grammatiken. Geben Sie eine rechtslineare Grammatik $G_{\circ}$ an, die genau $L(G) \circ L(G')$ erzeugt. (NK2017)
- Seien G und G' kontextfreie Grammatiken. Geben Sie eine kontextfreie Grammatik $G_{\cup}$ an, die genau $L(G) \cup L(G')$ erzeugt. (HK2017)

Sprachen mit festen Schranken / endliche Sprachen.

Trickfrage: feste Exponenten bzw. obere Schranke $\Rightarrow$ die Sprache ist endlich bzw. regulär, hier nicht auf das Pumping-Lemma hereinfallen!

- $\{\bar{x} \in \{\underline{a}, \underline{b}\}^* \mid \bar{x} = a^n b^n \text{ mit } n \in \mathbb{N}\} \cup \{\bar{x} \in \{\underline{c}, \underline{d}\}^* \mid |\bar{x}| \leq 42\}$ (HK2017)
- $\{\bar{x} \in \{\underline{a}, \dots, \underline{z}\}^* \mid \exists n : |\bar{x}| = 42^n\}$ (PK2021)
- $\{a^{2025} b^{2025} c^k \mid k \in \mathbb{N}_0, k \leq 2025\}$ (PK2025)

Warnung 2.1

- **Endlich $\Rightarrow$ regulär.** $a^{2025} b^{2025} c^k, k \leq 2025$ hat nur endlich viele Wörter $\Rightarrow$ regulär (und damit kontextfrei). Die festen 2025 sind Konstanten, keine Variablen!
- Das Pumping-Lemma ist **nur notwendig**, nicht hinreichend. Es beweist Nicht-Regularität, **niemals** Regularität.
- "Regulär $\Rightarrow$ nach dem Schnitt bestimmen. $\{n = k\} \cap \{\dots\}$ erzwingt oft $n = m = k$, was typischerweise nicht kontextfrei ist.

2.1.2 | Endliche Automaten

NEA aus rechtslinearer Grammatik konstruieren (als Diagramm).

Nichtterminale $\rightarrow$ Zustände, ein zusätzlicher Finalzustand f für Regeln $S ::= a$.

- Geben Sie zu folgender rechtslinearer Grammatik einen äquivalenten NEA (als Diagramm) an:
 - $G = (\{S_0, S_1\}, \{a, b, c\}, \{S_0 ::= bS_0 \mid aS_1 \mid bS_1 \mid a, S_1 ::= aS_0 \mid b\}, S_0)$ (PK2021)
 - $G_1 = (\{S_0, S_1\}, \{a, b\}, \{S_0 ::= aS_1 \mid bS_0 \mid bS_1 \mid a, S_1 ::= aS_0 \mid b\}, S_0)$ (PK2023)
 - $G := (\{S_0, S_1, S_2, S_3\}, \{a, b, c\}, R, S_0)$ mit
 $R := \{S_0 ::= cS_0 \mid c \mid aS_1 \mid bS_1, S_1 ::= cS_1 \mid aS_2 \mid bS_2, S_2 ::= cS_2 \mid aS_3 \mid bS_3 \mid a, S_3 ::= cS_3 \mid c\}$ (NK2023)
 - $G := (\{S_0, S_1, S_2\}, \{a, b, c\}, R, S_0)$ mit
 $R := \{S_0 ::= aS_0 \mid cS_0 \mid aS_1, S_1 ::= cS_1 \mid aS_2 \mid bS_2, S_2 ::= aS_2 \mid bS_2 \mid a \mid b\}$ (PK2025)
- Betrachten Sie die rechtslineare Grammatik G . Vervollständigen Sie gemäß der Konstruktion in der Vorlesung folgenden NEA, der genau die Sprache erkennen soll, die die Grammatik erzeugt.
 - $G = (\{A_0, A_1, B_0, B_1\}, \{a, b\}, \{A_0 ::= aA_1 \mid B_0, A_1 ::= aA_0 \mid B_1 \mid a, B_0 ::= bB_1 \mid \epsilon, B_1 ::= bB_0\}, A_0)$ (HK2019)
 - $G = (\{A_0, A_1, B_0, B_1\}, \{a, b\}, \{A_0 ::= aA_1 \mid B_0, A_1 ::= aA_0 \mid B_1 \mid a, B_0 ::= bB_1 \mid \epsilon, B_1 ::= bB_0\}, A_0)$ (NK2019)
- Geben Sie einen

Potenzmengenkonstruktion NEA $\rightarrow$ DEA.

Fast jede Klausur...Mal als Diagramm, mal als Übergangstabelle. Müllzustand $\emptyset$ mitnehmen. Zustände = Teilmengen

- Geben Sie zu folgendem NEA einen DEA an (Tupel oder Diagramm bzw. vollständige Übergangstabelle), der die gleiche Sprache akzeptiert.

- (HK2017) NEA über $\{a, b\}$, % Diagramm: $\Phi = \{q_0, q_1\}$, Start q_0 ; $q_0 \xrightarrow{b} q_0$, $q_0 \xrightarrow{a,b} q_1$, $q_1 \xrightarrow{a} q_1$, $q_1 \xrightarrow{b} q_0$; $F = \{q_1\}$
- (NK2017) NEA über $\{a, b\}$, % Diagramm: $\Phi = \{q_0, q_1\}$, Start q_0 ; $q_0 \xrightarrow{a} q_0$, $q_0 \xrightarrow{b} q_1$, $q_1 \xrightarrow{b} q_1$, $q_1 \xrightarrow{a,b} q_0$
- (PK2021) NEA über $\{a, b\}$, % Diagramm: $\Phi = \{q_0, q_1, q_2\}$, Start q_0 ; $q_0 \xrightarrow{a} q_2$, $q_0 \xrightarrow{a} q_1$, $q_1 \xrightarrow{a} q_0$, $q_1 \xrightarrow{a,b} q_2$, $q_2 \xrightarrow{b} q_2$
- (PK2023) DEA aus dem NEA von Teilaufgabe a) (Grammatik G_1)
- (NK2023) NEA A über $\{a, b\}$, % Diagramm: Start q_0 ;
 $\Delta: \delta(\{q_0\}, a) = \{q_1, q_2\}$, $\delta(\{q_0\}, b) = \{q_0\}$, $\delta(\{q_1\}, a) = \{q_0\}$, $\delta(\{q_1\}, b) = \{q_2\}$, $\delta(\{q_2\}, a) = \emptyset$, $\delta(\{q_2\}, b) = \emptyset$;
- (PK2025) DEA aus dem NEA von Teilaufgabe a) (6 Pkt., als Diagramm)

DEA direkt für eine gegebene Sprache angeben.

z. B. “ $\#_a(x)$ ungerade” oder “ $\#_a(x) \equiv 1 \pmod{4}$ ” aka Zähl-Automat modulo k .

- Geben Sie einen DEA (als Automatendiagramm) an für die Sprache:

- $\{x \in \{a, b, c\}^* \mid \#_a(x) \text{ ist ungerade}\}$ (HK2017)
- $\{x \in \{a, b\}^* \mid \#_a(x) \text{ ist mit Rest 1 durch 4 teilbar}\}$ (NK2017)

Komplement-Automat konstruieren.

Finalzustände tauschen nur beim vollständigen DEA korrekt!

- Konstruieren Sie einen NEA A_C (als Diagramm) mit $L(A_C) = L(A)^C$.
 □ A % Diagramm über $\{a, b, c\}$, $\Phi = \{q_0, q_1, q_2, q_3\}$, Start/Final q_0 ; $q_0 \xrightarrow{a,c} q_1$, $q_1 \xrightarrow{a} q_0$, $q_1 \xrightarrow{b} q_1$, $q_1 \xrightarrow{c} q_2$, $q_2 \xrightarrow{b} q_2$, $q_2 \xrightarrow{c} q_1$, $q_2 \xrightarrow{a} q_3$, $q_2 \xrightarrow{b} q_2$, $q_3 \xrightarrow{a,b,c} q_3$ (PK2021)
- Gegeben ein endlicher Automat $A = (\Phi, \Sigma, \Delta, q_0, F)$. Betrachte $A_C = (\Phi, \Sigma, \Delta, q_0, \Phi \setminus F)$. Unter welcher Annahme gilt $L(A_C) = L(A)^C$? Warum ist diese Annahme notwendig? (PK2023)
- Gegeben ein DEA $A = (\Phi, \Sigma, \delta, q_0, F)$. Konstruieren Sie einen DEA (in Tupelnotation), der genau das Komplement von $L(A)$ akzeptiert. Begründen Sie. (HK2019) (NK2019)

Kleene-Stern $L(A)^*$ bzw. $L(A) \cdot \{a\}$ konstruieren.

ε -Kanten von Finalzuständen zurück zum (neuen) Start; neuer akzeptierender Startzustand.

Konkatenation $L(A_1) \cdot L(A_2)$ / Vereinigung $L(A_1) \cup L(A_2)$.

Diagramm und allgemeine Tupel-Konstruktion (disjunkte Zustandsmengen $\Phi_1 \cap \Phi_2 = \emptyset$).

- Wie kann man allgemein zu einem NEA A einen NEA A' konstruieren mit $L(A') = (L(A))^*$? Demonstrieren Sie das Vorgehen am NEA aus Teilaufgabe b). (HK2017)
- Wie kann man allgemein zu einem NEA A einen NEA A' konstruieren mit $L(A') = L(A) \cdot \{a\}$? Demonstrieren Sie am NEA aus Teilaufgabe b). (NK2017)

- Konstruieren Sie einen NEA A_* mit $L(A_*) = L(A)^*$ (in Tupelnotation). Erläutern Sie, wie A_* ein Wort $\overline{w_1} \dots \overline{w_n}$ mit $\overline{w_i} \in L(A)$ ableitet (explizite Rechnung).
 - für allgemeines $A = (\Phi, \Sigma, \Delta, q_0, F)$ (PK2023)
 - NEA für $L(A)^*$ (Tupelnotation), Begründung (NK2019)

Äquivalenz Grammatik $\leftrightarrow$ Automat beweisen/widerlegen.

$L(G_1) = L(G_2)$? oder $L(A) \subsetneq L(G)$? Gegenbeispielwort finden oder Inklusion beider Richtungen.

- Beweisen oder widerlegen Sie, dass die von G_2 erzeugte Sprache mit der von G_1 übereinstimmt ($L(G_1) = L(G_2)$), beide als Sprachen über $\{a, b, c\}$:
 - $G_2 = (\{S'_0, S'_1, S'_2, S'_3\}, \{a, b, c\}, R, S'_0)$,
 $R := \{S'_0 ::= aS'_1 \mid bS'_0 \mid cS'_2 \mid a, S'_1 ::= aS'_0 \mid bS'_0 \mid cS'_2 \mid b, S'_2 ::= aS'_3, S'_3 ::= aS'_2 \mid bS'_2\}$ (PK2023)
- Beweisen oder widerlegen Sie: Die von der Grammatik G' erzeugte Sprache ist die vom Automaten A (aus Teilaufgabe c) akzeptierte Sprache, d.h. $L(A) = L(G')$.
 - $G' = (\{S'_0, S'_1, S'_2\}, \{a, b\}, \{S'_0 ::= aS'_1 \mid aS'_2 \mid bS'_0 \mid a, S'_1 ::= aS'_0 \mid bS'_2 \mid b, S'_2 ::= bS'_1\}, S'_0)$ (NK2023)
- Beweisen Sie: Die von der Grammatik G erzeugte Sprache steht in echter Inklusion zur vom Automaten A_3 akzeptierten Sprache, d.h. $L(A_3) \subsetneq L(G)$.
 - $G = (\{S'_0, S'_1, S'_2, S'_3\}, \{a, b, c\}, R, S'_0)$,
 $R := \{S'_0 ::= bS'_0 \mid aS'_1 \mid bS'_1 \mid aS'_2 \mid S'_3, S'_1 ::= aS'_0 \mid b, S'_2 ::= aS'_1 \mid b, S'_3 ::= S'_3 \mid \epsilon\}$.
 A_3 % Diagramm über $\{a, b\}$: q_0, q_1, q_2 mit Kanten b, a, a, b, b . (PK2025)
- Geben Sie zu dem NEA aus Aufgabenteil c eine rechtslineare Grammatik an, die genau die vom NEA akzeptierte Sprache erzeugt, und erzeugen Sie mit dieser Grammatik das Wort aabbb. (PK2021) (NEA A aus 2c, s. o.)

Deterministischen Ersatzakzeptor vervollständigen.

Schon lange her ... vermutlich nicht wieder?

- Betrachten Sie den NEA auf der linken Seite. Vervollständigen Sie gemäß der Konstruktion aus der Vorlesung den deterministischen Ersatzakzeptor (rechts vorgegebene Zustände).
 - NEA % Diagramm: $q_0 \xrightarrow{a} q_0, q_0 \xrightarrow{b} q_1, q_1 \xrightarrow{a} q_1, q_1 \xrightarrow{\epsilon} q_0$; vorgegebene DEA-Zustände $\{q_1\}, \{q_0\}, \{q_0, q_1\}$ (HK2019)
 - NEA % Diagramm: $q_0 \xrightarrow{a} q_0, q_0 \xrightarrow{b} q_1, q_1 \xrightarrow{b} q_2, q_2 \xrightarrow{a} q_2, q_2 \xrightarrow{\epsilon} q_0$; vorgegebene DEA-Zustände $\{q_0\}, \{q_1\}, \{\}, \{q_0, q_2\}, \{q_2\}$ (NK2019)

2.1.3 | Kontextfreie Grammatiken & LR/Kellerautomaten

Eindeutigkeit

Eindeutig = jedes Wort hat genau einen Strukturbaum.

- Ist die Grammatik eindeutig? Begründen Sie.
 - $G = (\{S\}, \{a, b\}, \{S ::= aSSa \mid b \mid a\}, S)$ (HK2017)
 - $G = (\{S, B\}, \{a, b\}, \{S ::= aBSa \mid \epsilon, B ::= bS \mid Sb\}, S)$ (NK2017)
 - $G = (\{S, B\}, \{a, b, c\}, \{S ::= aSc \mid B, B ::= bB \mid \epsilon\}, S)$ (HK2019)
 - $G = (\{S, B\}, \{a, b, c\}, \{S ::= aSc \mid B, B ::= bB \mid \epsilon\}, S)$ (NK2019)
 - $G = (\{S, B\}, \{a, b, c\}, \{S ::= aScB \mid B, B ::= bB \mid b\}, S)$ (PK2021)
 - $G = (\{S\}, \{a, b, c\}, \{S ::= bSSb \mid c \mid b\}, S)$ (PK2025)

Regeln des zur Grammatik gehörenden LR-Automaten explizit angeben.

Shift-Regeln (Terminal lesen) und Reduce-Regeln (pro Grammatikregel).

- Geben Sie die Regeln des zur Grammatik G gehörenden LR-Automaten an

- ☐ $G = (\{S\}, \{a, b\}, \{S ::= aSSa \mid b \mid a\}, S)$ (HK2017)
- ☐ $G = (\{S, B\}, \{a, b\}, \{S ::= aBSa \mid \epsilon, B ::= bS \mid Sb\}, S)$ (NK2017)
- ☐ $G = (\{S, B\}, \{a, b, c\}, \{S ::= aSc \mid B, B ::= bB \mid \epsilon\}, S)$ (HK2019)
- ☐ $G = (\{S, B\}, \{a, b, c\}, \{S ::= aSc \mid B, B ::= bB \mid \epsilon\}, S)$ (NK2019)
- ☐ $G = (\{S, B\}, \{a, b, c\}, \{S ::= aScB \mid B, B ::= bB \mid b\}, S)$ (PK2021)
- ☐ $G = (\{S\}, \{(\cdot, a, \cdot)\}, \{S ::= (SS) \mid aS \mid a\}, S)$ (PK2023)
- ☐ $G = (\{S\}, \{a, b\}, \{S ::= aSSa \mid a \mid b\}, S)$ (NK2023)
- ☐ $G = (\{S\}, \{a, b, c\}, \{S ::= bSSb \mid c \mid b\}, S)$ (PK2025)

Ein Wort mit dem LR-Automaten rechnen und den Strukturbaum angeben.

Jeden Shift/Reduce-Schritt (Kellerinhalt) angeben.

- Rechnen Sie das Wort w und geben Sie ggf. den Strukturbaum an:

- ☐ $w = \underline{abba}$ (mit Strukturbaum) (HK2017)
- ☐ $w = \underline{aba}$ (mit Strukturbaum) (NK2017)
- ☐ $w = \underline{aabbcbcb}$ (PK2021)
- ☐ $w = \underline{(a(aaa))}$ (mit Strukturbaum) (PK2023)
- ☐ $w = \underline{abba}$ (mit Strukturbaum) (NK2023)
- ☐ $w = \underline{bcbccbb}$ (jeden Schritt der Rechnung angeben) (PK2025)

- Geben Sie einen Strukturbaum für das Wort in dieser Grammatik an:

- ☐ $w = \underline{aabcc}$ (HK2019)
- ☐ $w = \underline{aabcc}$ (NK2019)

CYK-Algorithmus (Cocke-Kasami-Younger). Grammatik in CNF.

- Prüfen Sie mit dem CYK-Algorithmus, ob $w \in L(G)$ (Grammatik aus Teilaufgabe c bzw. gegeben):

- ☐ $w = (a(aaa))$ für $G = (\{S\}, \{(\cdot, a, \cdot)\}, \{S ::= (SS) \mid aS \mid a\}, S)$ (PK2023)
- ☐ $G_1 = (\{A, B, C\}, \{a, b, c\}, \{A ::= AB \mid AC \mid BC \mid a, B ::= BB \mid BC \mid b, C ::= c\}, A)$; prüfe $\underline{acb} \in L(G_1)$ (NK2023)
- ☐ $G_{\text{CNF}} = (\{A, B, C\}, \{a, b, c\}, \{A ::= AB \mid BC \mid a, B ::= BB \mid b, C ::= AC \mid CC \mid c\}, B)$;
(i) zeigen Sie $w = ccabbc \notin L(G_{\text{CNF}})$; (ii) für welchen Startzustand würde w akzeptiert? (PK2025)

Kellerautomat vervollständigen und Rechnung angeben.

Regeln $a q_0 b ::= \overline{w} q_1$ graphisch als $(a, b)/\overline{w}$.

- Vervollständigen Sie den (indeterministischen) Kellerautomaten, der dieselbe Sprache erkennt wie G , und geben Sie die Rechnung für das Wort an.

- ☐ Zustände q_a, q_b, q_c, f (Diagramm), Rechnung für $\underline{aabcc}$ (HK2019)
- ☐ Regelschablonen $(\#, a)/_, (\#, b)/_, (a, a)/_, (a, b)/_, (a, c)/_$ und $\#/\epsilon$ nach f ; Rechnung für $\underline{aabcc}$ (NK2019)

Induktionsbeweis über Wörter / Sprachcharakterisierung.

Induktion über den Ableitungsbaum bzw. die Regelanwendungen.

■ Zeigen Sie durch Induktion, dass für alle $\bar{x} \in L(G)$ gilt:

$$\square \#_a(\bar{x}) \geq \#_b(\bar{x}) \quad \text{oder} \quad \bar{x} = b \quad (\text{HK2017})$$

$$\square \#_a(\bar{x}) \geq \#_b(\bar{x}) \quad (\text{NK2017})$$

$$\square \#_b(\bar{w}) \leq \#_a(\bar{w}) + 1 \quad (\text{NK2023})$$

■ Zeigen Sie, dass für jedes Wort die Anzahl der b 's mindestens um 1 größer ist als die Anzahl der a 's. (PK2021)

■ Betrachten Sie die Sprache $L' = \{\bar{w} \in \{(a, a)\}^* \mid \#_a(\bar{x}) \geq \#_b(\bar{x}) \text{ und } \#_a(\bar{y}) \leq \#_b(\bar{y}) \text{ für alle Zerlegungen } \bar{w} = \bar{x}\bar{y}\}$. Zeigen Sie $L(G) \subsetneq L'$. (PK2023)

2.1.4 | Probleme / Berechenbarkeit & Reduktion**2.1.5 | Multiple-Choice****Sprachen & Automaten**

Aussage	Begründung
Jede endliche Sprache ist regulär.	W endlich $\Rightarrow$ RegExp als Aufzählung.
$\exists$ Paar kontextfreier L_1, L_2 mit $L_1 \cup L_2$ kontextfrei.	W CFL sogar immer unter $\cup$ abgeschl.
$\exists$ Paar kontextfreier L_1, L_2 mit $L_1 \cap L_2$ kontextfrei.	W "es gibt ein Paar" (z. B. regulär); nicht für alle.
Kontextfreie Sprachen sind gegen Schnitt abgeschlossen.	F Gegenbsp. $a^n b^n c^n$.
$\exists$ kontextsensitive L mit L^C nicht kontextsensitiv.	F kontextsensitiv unter Kompl. abgeschl.
Reg. Sprache unendlich $\iff \exists$ Wort $n_0 \leq w < 2n_0$ ($n_0 = \# \text{Zust.}$).	W Standard-Charakterisierung.
Reg. Sprache unendlich, sobald ein Wort der Länge $\geq n_0$ existiert.	W Zustand wiederholt sich $\Rightarrow$ pumpbar.
Reg. Grammatik mit n Nichtterm. $\Rightarrow$ jeder DEA $\geq n$ Zustände.	F minimaler DEA kann kleiner sein.
LR-/DPDA-Sprachen sind gegen Komplement abgeschlossen.	W DCFL unter Komplement abgeschl.
DPDAs/PDAs gegen Schnitt abgeschlossen.	F schon CFL nicht unter $\cap$.
DPDAs gegen Schnitt mit DEA abgeschlossen.	W $\text{DCFL} \cap \text{regulär} = \text{DCFL}$.
$\{a^n b^m c^n d \mid m, n\}$ durch DPDA erkennbar.	W deterministisch: a push, c pop, d prüfen.
Für kontextfreie Sprachen ist das Wortproblem nicht entscheidbar.	F CYK entscheidet es ($\mathcal{O}(n^3)$).
$G = (\{S\}, \{1, 2\}, \{S ::= S1S \mid 2\}, S)$ ist eindeutig.	F z. B. "21212" hat zwei Bäume.
Jede links-/rechtslineare Grammatik ist eindeutig.	F reguläre Grammatiken können mehrdeutig sein.
$G : S ::= SS \mid (S) \mid \varepsilon$ ist nicht eindeutig.	W ε/SS erzeugen mehrere Bäume.

Berechenbarkeit & Komplexität

Aussage		Begründung
Probleme in NP sind entscheidbar.	W	$NP \subseteq$ entscheidbar.
Ein unentscheidbares Problem kann nicht in NP sein.	W	Kontraposition von oben.
Ein unentscheidbares Problem kann nicht NP-hart sein.	F	z. B. Halteproblem ist NP-hart.
Rekursiv aufzählbare Sprachen werden von det. TM akzeptiert.	W	det. TM = nichtdet. TM (r. a.).
$\{(a, b, c) \mid \varphi_{a+b}(c) \downarrow\}$ ist formale Regelsprache (r. a.).	W	Halteproblem ist semi-entscheidbar.
$\{(a, b, c) \mid \varphi_{a+b}(c) \downarrow\}$ ist entscheidbar.	F	Halteproblem unentscheidbar.
$\{(a, b) \mid \varphi_a(b) \uparrow\}$ ist rekursiv aufzählbar.	F	Komplement des Halteproblems, nicht r. a.
Es gibt abzählbar viele verschiedene Turingmaschinen.	W	jede TM endlich kodierbar.
Es gibt überabzählbar viele verschiedene Turingmaschinen.	F	siehe oben.
$P \subseteq PSPACE$.	W	Standardinklusion.
$P \subset NP \cap co-NP$.	W(?)	$\subseteq$ gilt; echte Inklusion offen.
Menge der entscheidbaren Probleme gegen Schnitt abgeschlossen.	W	zwei Entscheider "und"-verknüpfen.
Für beliebige entscheidbare L_1, L_2 ist " $L_1 \subseteq L_2$ " entscheidbar.	F	reduziert von Leereheit/Halteproblem.
Für beliebige kontextfreie L_1, L_2 ist " $L_1 \subseteq L_2$ " entscheidbar.	F	CFL-Inklusion unentscheidbar.
Entscheidbar, ob eine TM nach $\leq t$ Schritten hält.	W	einfach t Schritte simulieren.
Wortproblem für kontextsensitive Sprachen ist entscheidbar.	W	LBA / Chomsky-1.
Formale Regelsprachen abgeschl. gegen $\cap, \cup, *$.	F	Komplement fehlt (r. a. nicht unter $\cup$).

Reduktion, NP-Vollständigkeit, Gödelisierung

Aussage		Begründung
A NP-vollständig $\iff A \leq_p SAT$ und $SAT \leq_p A$.	W	$A \leq_p SAT \Rightarrow A \in NP$; $SAT \leq_p A \Rightarrow A$ NP-hart.
A NP-vollständig $\iff SAT \leq_p A$.	F	$A \in NP$ fehlt (das wäre nur NP-hart).
A NP-hart $\iff A \leq_p SAT$.	F	Richtung falsch (müsste $SAT \leq_p A$).
A NP-hart $\iff A \leq_p ILP \wedge ILP \leq_p A$.	F	beschreibt NP-vollständig, nicht NP-hart.
Für jede reguläre Sprache A gilt $A \leq_p 3-SAT$.	W	regulär $\subseteq P \subseteq NP$; 3-SAT NP-vollst.
$A \in \text{Chomsky-2} \Rightarrow 2-SAT \leq_p A$.	F	Gegenbsp. $A = \emptyset$ (kein Ziel für Ja-Instanzen).
Longest-Path (Pfad Länge $\geq k$, knotendisjunkt) ist NP-hart.	W	Hamiltonpfad als Spezialfall.
Erreichbarkeit ($\exists$ Pfad $u-v$) ist NP-hart.	F	in P (BFS/DFS).
Fair teilen (Werte auf k Personen gleich) ist NP-vollständig.	W	Partition/Bin-Packing.
Bier auf Gläser ohne Überlauf verteilen.	W	Bin-Packing $\Rightarrow$ NP-hart.
Wunsch-Zimmernachbar (je 1 Wunsch) erfüllbar?	F	Matching $\Rightarrow$ in P.
$\{1, \dots, 5\}^* \rightarrow \mathbb{N}$, $(n_1, \dots, n_t) \mapsto \sum n_i 10^i$ ist Gödelisierung.	W(?)	Ziffern $< 10 \Rightarrow$ injektiv; def.-abhängig (nicht surjektiv).
$\mathbb{N}^* \rightarrow \mathbb{N}$, $(n_1, \dots, n_t) \mapsto \sum n_i 10^i$ ist Gödelisierung.	F	unbeschr. $n_i \Rightarrow$ Überträge, nicht injektiv.
Eine Gödelisierung muss polynomiell berechenbar sein.	F	nur berechenbar & injektiv nötig.
Universelle TM mit 42 Zuständen und 42 Bandzeichen.	W	UTMs existieren, 42×42 reicht üppig.
TM mit 42 Zuständen simulierbar durch TM mit wenigen Zuständen.	W(?)	Zustands-/Zeichen-Tradeoff; Skript prüfen.
Universelle TM mit einem Zustand.	(?)	umstritten ... im Skript / Tutorium klären.

Backmatter

Definitions

Σ^m Menge der Wörter der Länge m über Σ . 6, 7

Σ^* Menge aller Wörter über Σ . 6, 8

Anzahl der Vorkommnisse des Symbols in einem Wort Die Anzahl der Vorkommnisse des Symbols a_i im Wort $\bar{w}$. 6

erzeugte Sprache todo. 8

formale Regelsprache todo. 8

formale Sprache todo. 8

Grammatik todo. 8

Konkatenation Eine Verkettung zweier Wörter $\bar{w} = a_{i_1} \dots a_{i_m}$ und $\bar{v} = a_{i_1} \dots a_{i_l}$ zu

$$\overline{wv} = a_{i_1} \dots a_{i_m} a_{i_1} \dots a_{i_l}$$

. 6

leeres Wort Wort der Länge 0, notiert durch ϵ . 6, 7

lineare Grammatik todo. 8

Monoid Assoziative Strukturen, die ein neutrales Element haben und für die eine assoziative Operation mit Einselement existiert. 7

Postfix $\bar{v}$ von $\bar{u}$ Wenn $\bar{u} = \bar{wv}$. 6

Potenz todo. 8

Präfix Wenn $\bar{u} = \bar{wv}$. 6

Symbol todo. 6

Verkettung todo. 8

Wort Eine endliche Folge von Elementen aus einem Alphabet:

$$\bar{w} = a_{i_1} \dots a_{i_m}, \quad a_{i_j} \in \Sigma, \quad m > 0$$

Dabei ist $m = \bar{w}$ die Länge des Wortes und ein Wort der Länge 0 ist das leere Wort ϵ . 6

Äquivalenz todo. 8

Literatur

- [1] *Grundlagen Theoretischer Informatik*. Vorlesungsskript SS2026.